

We have now seen two operational models for programming languages: small-step and large-step. In this lecture, we consider a different semantic model, called *denotational semantics*.

The idea in denotational semantics is to express the meaning of a program as the mathematical function that expresses what the program computes. We can think of an IMP program c as a function from stores to stores: given an initial store, the program produces a final store. For example, the program $\text{foo} := \text{bar} + 1$ can be thought of as a function that when given an input store σ , produces a final store σ' that is identical to σ except that it maps foo to the integer $\sigma(\text{bar}) + 1$; that is, $\sigma' = \sigma[\text{foo} \mapsto \sigma(\text{bar}) + 1]$. We will model programs as functions from input stores to output stores. As opposed to operational models, which tell us *how* programs execute, the denotational model shows us *what* programs compute.

1 A Denotational Semantics for IMP

For each program c , we write $C[[c]]$ for the *denotation* of c , that is, the mathematical function that c represents:

$$C[[c]] : \text{Store} \rightarrow \text{Store}.$$

Note that $C[[c]]$ is actually a partial function (as opposed to a total function), both because the store may not be defined on the free variables of the program and because program may not terminate for certain input stores. The function $C[[c]]$ is not defined for non-terminating programs as they have no corresponding output stores.

We will write $C[[c]]\sigma$ for the result of applying the function $C[[c]]$ to the store σ . That is, if f is the function that $C[[c]]$ denotes, then we write $C[[c]]\sigma$ to mean the same thing as $f(\sigma)$.

We must also model expressions as functions, this time from stores to the values they represent. We will write $\mathcal{A}[[a]]$ for the denotation of arithmetic expression a , and $\mathcal{B}[[b]]$ for the denotation of boolean expression b .

$$\mathcal{A}[[a]] : \text{Store} \rightarrow \text{Int}$$

$$\mathcal{B}[[b]] : \text{Store} \rightarrow \{\text{true}, \text{false}\}$$

Now we want to define these functions. To make it easier to write down these definitions, we will describe (partial) functions using sets of pairs. More precisely, we will represent a partial map $f : A \rightarrow B$ as a set of pairs $F = \{(a, b) \mid a \in A \text{ and } b = f(a) \in B\}$ such that, for each $a \in A$, there is at most one pair of the form $(a, _)$ in the set. Hence $(a, b) \in F$ is the same as $b = f(a)$.

We can now define denotations for IMP. We start with the denotations of expressions:

$$\begin{aligned}\mathcal{A}[\![n]\!] &= \{(\sigma, n)\} \\ \mathcal{A}[\![x]\!] &= \{(\sigma, \sigma(x))\} \\ \mathcal{A}[\![a_1 + a_2]\!] &= \{(\sigma, n) \mid (\sigma, n_1) \in \mathcal{A}[\![a_1]\!] \wedge (\sigma, n_2) \in \mathcal{A}[\![a_2]\!] \wedge n = n_1 + n_2\}\end{aligned}$$

$$\begin{aligned}\mathcal{B}[\![\mathbf{true}]\!] &= \{(\sigma, \mathbf{true})\} \\ \mathcal{B}[\![\mathbf{false}]\!] &= \{(\sigma, \mathbf{false})\} \\ \mathcal{B}[\![a_1 < a_2]\!] &= \{(\sigma, \mathbf{true}) \mid (\sigma, n_1) \in \mathcal{A}[\![a_1]\!] \wedge (\sigma, n_2) \in \mathcal{A}[\![a_2]\!] \wedge n_1 < n_2\} \cup \\ &\quad \{(\sigma, \mathbf{false}) \mid (\sigma, n_1) \in \mathcal{A}[\![a_1]\!] \wedge (\sigma, n_2) \in \mathcal{A}[\![a_2]\!] \wedge n_1 \geq n_2\}\end{aligned}$$

The denotations for commands are as follows:

$$\begin{aligned}C[\![\mathbf{skip}]\!] &= \{(\sigma, \sigma)\} \\ C[\![x := a]\!] &= \{(\sigma, \sigma[x \mapsto n]) \mid (\sigma, n) \in \mathcal{A}[\![a]\!]\} \\ C[\![c_1; c_2]\!] &= \{(\sigma, \sigma') \mid \exists \sigma''. ((\sigma, \sigma'') \in C[\![c_1]\!] \wedge (\sigma'', \sigma') \in C[\![c_2]\!])\}\end{aligned}$$

Note that $C[\![c_1; c_2]\!] = C[\![c_2]\!] \circ C[\![c_1]\!]$, where $\circ$ is the composition of relations, defined as follows: if $R_1 \subseteq A \times B$ and $R_2 \subseteq B \times C$ then $R_2 \circ R_1 \subseteq A \times C$ is $R_2 \circ R_1 = \{(a, c) \mid \exists b \in B. (a, b) \in R_1 \wedge (b, c) \in R_2\}$. If $C[\![c_1]\!]$ and $C[\![c_2]\!]$ are total functions, then $\circ$ is function composition.

$$\begin{aligned}C[\![\mathbf{if } b \mathbf{ then } c_1 \mathbf{ else } c_2]\!] &= \{(\sigma, \sigma') \mid (\sigma, \mathbf{true}) \in \mathcal{B}[\![b]\!] \wedge (\sigma, \sigma') \in C[\![c_1]\!]\} \cup \\ &\quad \{(\sigma, \sigma') \mid (\sigma, \mathbf{false}) \in \mathcal{B}[\![b]\!] \wedge (\sigma, \sigma') \in C[\![c_2]\!]\} \\ C[\![\mathbf{while } b \mathbf{ do } c]\!] &= \{(\sigma, \sigma) \mid (\sigma, \mathbf{false}) \in \mathcal{B}[\![b]\!]\} \cup \\ &\quad \{(\sigma, \sigma') \mid (\sigma, \mathbf{true}) \in \mathcal{B}[\![b]\!] \wedge \exists \sigma''. ((\sigma, \sigma'') \in C[\![c]\!] \wedge (\sigma'', \sigma') \in C[\![\mathbf{while } b \mathbf{ do } c]\!])\}\end{aligned}$$

But now we have a problem: the last “definition” is not really a definition because it expresses $C[\![\mathbf{while } b \mathbf{ do } c]\!]$ in terms of itself! This is not a definition but a recursive equation. What we want is the solution to this equation.

2 Fixed Points

We gave a recursive equation that the function $C[\![\mathbf{while } b \mathbf{ do } c]\!]$ must satisfy. To understand some of the issues involved, let’s consider a simpler example. Consider the following equation for a function $f : \mathbb{N} \rightarrow \mathbb{N}$.

$$f(x) = \begin{cases} 0 & \text{if } x = 0 \\ f(x-1) + 2x - 1 & \text{otherwise} \end{cases} \quad (1)$$

This is not a definition for f , but rather an equation that we want f to satisfy. What function, or functions, satisfy this equation for f ? The only solution to this equation is the function $f(x) = x^2$.

In general, there may be no solutions for a recursive equation (e.g., there are no functions $g : \mathbb{N} \rightarrow \mathbb{N}$ that satisfy the recursive equation $g(x) = g(x) + 1$), or multiple solutions (e.g., find two functions $g : \mathbb{R} \rightarrow \mathbb{R}$ that satisfy $g(x) = 4 \times g(\frac{x}{2})$).

We can compute solutions to such equations by building successive approximations. Each approximation is closer and closer to the solution. To solve the recursive equation for f , we start with the partial function $f_0 = \emptyset$ (i.e., f_0 is the empty relation; it is a partial function with the empty set for its domain). We compute successive approximations using the recursive equation.

$$\begin{aligned}
f_0 &= \emptyset \\
f_1 &= \begin{cases} 0 & \text{if } x = 0 \\ f_0(x-1) + 2x - 1 & \text{otherwise} \end{cases} \\
&= \{(0, 0)\} \\
f_2 &= \begin{cases} 0 & \text{if } x = 0 \\ f_1(x-1) + 2x - 1 & \text{otherwise} \end{cases} \\
&= \{(0, 0), (1, 1)\} \\
f_3 &= \begin{cases} 0 & \text{if } x = 0 \\ f_2(x-1) + 2x - 1 & \text{otherwise} \end{cases} \\
&= \{(0, 0), (1, 1), (2, 4)\} \\
&\vdots
\end{aligned}$$

This sequence of successive approximations f_i gradually builds the function $f(x) = x^2$.

We can model this process of successive approximations using a higher-order function F that takes one approximation f_k and returns the next approximation f_{k+1} :

$$F : (\mathbb{N} \rightarrow \mathbb{N}) \rightarrow (\mathbb{N} \rightarrow \mathbb{N})$$

where

$$(F(f))(x) = \begin{cases} 0 & \text{if } x = 0 \\ f(x-1) + 2x - 1 & \text{otherwise} \end{cases}$$

A solution to the recursive equation [1](#) is a function f such that $f = F(f)$. In general, given a function $F : A \rightarrow A$, we have that $a \in A$ is a *fixed point* of F if $F(a) = a$. We also write $a = \text{fix}(F)$ to indicate that a is a least fixed point of F .

So the solution to the recursive equation [1](#) is a fixed-point of the higher-order function F . We can compute this fixed point iteratively, starting with $f_0 = \emptyset$ and at each iteration computing $f_{k+1} = F(f_k)$. The fixed point is the limit of this process:

$$\begin{aligned}
f &= \text{fix}(F) \\
&= f_0 \cup f_1 \cup f_2 \cup f_3 \cup \dots \\
&= \emptyset \cup F(\emptyset) \cup F(F(\emptyset)) \cup F(F(F(\emptyset))) \cup \dots \\
&= \bigcup_{i \geq 0} F^i(\emptyset)
\end{aligned}$$

3 Denotation for Loops

We can now write the correct denotation case for **while** loops as the fixed point of a higher-order function:

$$\begin{aligned} C[\llbracket \text{while } b \text{ do } c \rrbracket] &= \text{fix}(F) \\ \text{where } F(f) &= \{(\sigma, \sigma) \mid (\sigma, \text{false}) \in \mathcal{B}[\llbracket b \rrbracket]\} \cup \\ &\quad \{(\sigma, \sigma') \mid (\sigma, \text{true}) \in \mathcal{B}[\llbracket b \rrbracket] \wedge \exists \sigma''. (\sigma, \sigma'') \in C[\llbracket c \rrbracket] \wedge (\sigma'', \sigma') \in f\} \end{aligned}$$

4 Kleene's Fixed Point Theorem

Next we'll prove Kleene's fixpoint theorem, which shows that the fixed point used to define the semantics of **while** commands exists.

Definition (Scott Continuity). A function F from U to U is said to be *Scott-continuous* if for every chain $X_1 \subseteq X_2 \subseteq \dots$ we have $F(\bigcup_i X_i) = \bigcup_i F(X_i)$.

It is not hard to show that if F is Scott continuous, then it is also monotonic—that is, $X \subseteq Y$ implies $F(X) \subseteq F(Y)$. The proof of this fact is left as an exercise.

Theorem (Kleene Fixpoint). *Let F be a Scott-continuous function. The least fixed point of F is $\bigcup_i F^i(\emptyset)$.*

Proof. Let $X = \bigcup_i F^i(\emptyset)$.

First, we will prove that X is a fixed point of F —that is, $F(X) = X$. We calculate as follows:

$$\begin{aligned} F(X) &= F(\bigcup_i F^i(\emptyset)) && \text{By definition of } X \\ &= \bigcup_i F(F^i(\emptyset)) && \text{By Scott continuity} \\ &= \bigcup_i F^{i+1}(\emptyset) \\ &= \emptyset \cup \bigcup_i F^{i+1}(\emptyset) \\ &= F^0(\emptyset) \cup \bigcup_i F^{i+1}(\emptyset) \\ &= \bigcup_i F^i(\emptyset) \\ &= X \end{aligned}$$

Second, we will show that X is the least fixed point of F . Suppose that Y is some other arbitrary fixed point of F .

By induction, we can easily show that $F^i(\emptyset) \subseteq Y$ for all i . For the base case, i is 0 and we trivially have $F^0(\emptyset) = \emptyset \subseteq Y$. For the inductive case, we assume that $F^i(\emptyset) \subseteq Y$ and prove that $F^{i+1}(\emptyset) \subseteq Y$. By our inductive hypothesis and the fact that F is monotone, we have that $F(F^i(\emptyset)) \subseteq F(Y)$. As Y is a fixed point we also have $F(Y) = Y$ and so $F^{i+1}(\emptyset) \subseteq Y$.

Then, since every element of the chain

$$F^0(\emptyset) \subseteq F^1(\emptyset) \subseteq \dots$$

is a subset of Y immediately we have that their union, $X = \bigcup_i F^i(\emptyset) \subseteq Y$. Hence, X is the least (with respect to $\subseteq$) fixed point of F . $\square$

5 Reasoning using Denotational Semantics

One of the main advantages of using denotational semantics compared to operational semantics is that proofs of equivalence can be carried out directly by simply calculating the denotations of programs and then arguing that they are identical. This is in contrast to operational techniques, where one must reason explicitly about low-level transitions and derivations involving ad hoc abstract machines.

As an example, to show that **skip**; c and c ; **skip** are equivalent, we can calculate as follows,

$$\begin{aligned} C[\llbracket \mathbf{skip}; c \rrbracket] &= \{(\sigma, \sigma'') \mid \exists \sigma'. (\sigma, \sigma') \in C[\llbracket \mathbf{skip} \rrbracket] \wedge (\sigma', \sigma'') \in C[\llbracket c \rrbracket]\} \\ &= \{(\sigma, \sigma'') \mid (\sigma, \sigma'') \in C[\llbracket c \rrbracket]\} \\ &= \{(\sigma, \sigma'') \mid \exists \sigma'. (\sigma, \sigma') \in C[\llbracket c \rrbracket] \wedge (\sigma', \sigma'') \in C[\llbracket \mathbf{skip} \rrbracket]\} \\ &= C[\llbracket c; \mathbf{skip} \rrbracket] \end{aligned}$$

using standard facts about partial functions, relations, and sets as convenient in the proof itself.

Next, consider the command $C[\llbracket \mathbf{while\ false\ do\ } c \rrbracket]$. By the definition of the denotational semantics, this is equal to $fix(F)$ where

$$\begin{aligned} F(f) &= \{(\sigma, \sigma) \mid (\sigma, \mathbf{false}) \in \mathcal{B}[\llbracket b \rrbracket]\} \cup \\ &\quad \{(\sigma, \sigma'') \mid (\sigma, \mathbf{true}) \in \mathcal{B}[\llbracket b \rrbracket] \wedge (\sigma, \sigma') \in C[\llbracket c \rrbracket] \wedge (\sigma', \sigma'') \in f\} \end{aligned}$$

By the Kleene fixpoint theorem we have that $fix(F) = \bigcup_i F^i(\emptyset)$. It is straightforward to show by induction that since the guard is **false**, for all i we have $F^i(\emptyset) = \{(\sigma, \sigma)\}$. It follows that $fix(F) = \{(\sigma, \sigma)\}$, which is just $C[\llbracket \mathbf{skip} \rrbracket]$.

As an exercise, calculate $C[\llbracket \mathbf{while\ true\ do\ skip} \rrbracket]$ using the same technique.